

**PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK
IMPLEMENTASI SISTEM KEAMANAN DAN INTEGRITAS DATA
PADA STUDI KASUS MINI BANK**



Disusun Oleh:
Haliya Isma Husna Putri Ahmadi
2511532002

Dosen Pengampu:
Nurfiah, S.ST,M.Kom.

Asisten Praktikum:
Ezza Andini

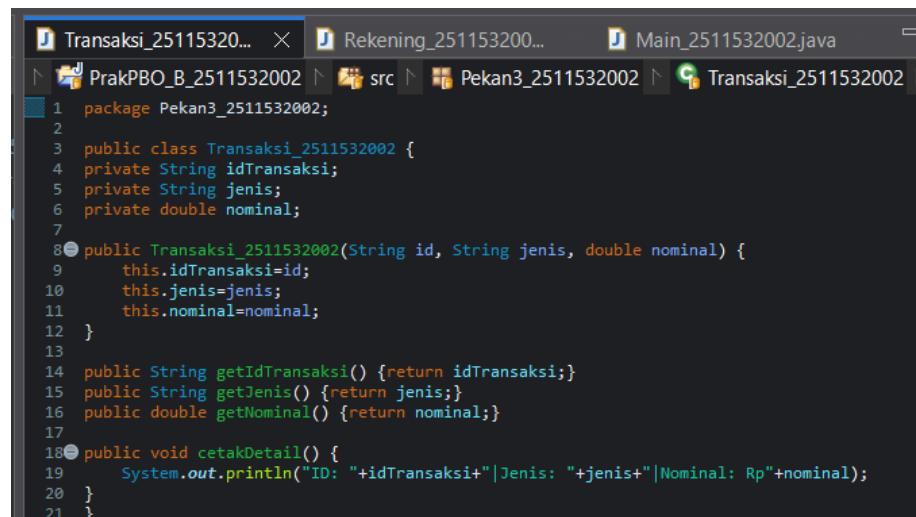
**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

A. LANGKAH KERJA

a) Membuat *Transaksi* menjadi *private*

1. Membuat *file Transaksi_2511532002.java* dalam *package Pekan3_2511532002* dari praktikum sebelumnya.
2. Mengubah kembali jenis atribut (*idTransaksi*, *jenis*, dan *nominal*) menjadi *private* sehingga tidak dapat diakses langsung dari luar *class*.
3. Membuat *getter* supaya nilainya bisa dibaca dari luar *class*, tapi tidak dibuatkan cara untuk mengubahnya.



```
1 package Pekan3_2511532002;
2
3 public class Transaksi_2511532002 {
4     private String idTransaksi;
5     private String jenis;
6     private double nominal;
7
8     public Transaksi_2511532002(String id, String jenis, double nominal) {
9         this.idTransaksi=id;
10        this.jenis=jenis;
11        this.nominal=nominal;
12    }
13
14    public String getIdTransaksi() {return idTransaksi;}
15    public String getJenis() {return jenis;}
16    public double getNominal() {return nominal;}
17
18    public void cetakDetail() {
19        System.out.println("ID: "+idTransaksi+"|Jenis: "+jenis+"|Nominal: Rp"+nominal);
20    }
21 }
```

Keterangan:

1) *private String idTransaks;*

private String jenis;

private double nominal;

Ketiga atribut dikunci dengan *access modifier private*.

Dimana pada sebelumnya atribut tidak memiliki *modifier* sehingga dapat diakses langsung dari luar *class*.

2) *Constructo Transaksi_2511532002(String id, String jenis, double nominal)*

Constructor untuk mengisi nilai ketiga atribut yaitu *id*, *jenis* dan *nominal*.

3) *getIdTransaksi()*, *getJenis()*, dan *getNominal()*

Getter agar class lain dapat membaca nilai atribut tersebut tanpa dapat mengubahnya.

4) *cetakDetail()*

Method yang menampilkan *id*, *jenis*, dan *nominal* transaksi.

b) Mengubah atribut pada *Rekening_2511532002* menjadi *private* dan menambahkan *PIN*

1. Buka kembali file *rekening_2511532002* dari praktikum sebelumnya, lalu mengubah atribut *nomorRekening*, *namaPemilik*, *saldo*, dan *riwayatTransaksi* menjadi *private*. Serta menambahkan atribut baru *private String pin* untuk menyimpan *PIN* nantinya.

```
public class Rekening_2511532002 {  
    private String nomorRekening;  
    private String namaPemilik;  
    private double saldo;  
    private String pin;  
  
    private ArrayList<Transaksi_2511532002> riwayatTransaksi;
```

2. Membuat fitur *PIN*, dengan memodifikasi *constructor* agar menerima parameter *pinAwal*, dimana dilakukan validasi apabila panjang *PIN* tepat 6 karakter maka *PIN* tersebut digunakan, dan jika tidak maka ditampilkan pesandan *PIN* otomatis diisi dengan nilai *default* “123456”.

```
public Rekening_2511532002(String nomor, String nama, double saldoAwal, String pinAwal) {  
    this.nomorRekening=nomor;  
    this.namaPemilik=nama;  
    this.saldo=saldoAwal;  
  
    //Validasi pin di dalam Constructor  
    if (pinAwal.length()==6) {  
        this.pin=pinAwal;  
    }else {  
        System.out.println("Peringatan: PIN Harus 6 digit! Menggunakan PIN Default 123456");  
        this.pin="123456";  
    }  
  
    this.riwayatTransaksi=new ArrayList<>();  
    System.out.println("Rekening atas nama "+ namaPemilik+" berhasil dibuat dengan saldo Rp"+saldo);  
}
```

3. Menambahkan *getter* untuk atribut yang diizinkan dibaca dari luar *class*, serta *method otentikasi()* yang membandingkan *PIN* yang *diinput* dengan *PIN* yang tersimpan.

```

//getter
public String getNomorRekening() {
    return nomorRekening;
}
public String getNamaPemilik() {
    return namaPemilik;
}
public double getSaldo() {
    return saldo;
}

//xxxxx pin
public boolean otentikasi(String pinInput) {
    return this.pin.equals(pinInput);
}

```

Keterangan:

- 1) *private String pin;*
Atribut baru untuk menyimpan *PIN* yang bersifat *private* karena merupakan data yang tidak boleh diakses langsung dari luar *class*.
- 2) Validasi *PIN* di dalam *constructor*
Pengecekan panjang *PIN* yang dilakukan di dalam *constructor*.
- 3) *getNomorRekening()*, *getNamaPemilik()*, dan *getSaldo()*
Getter hanya disediakan untuk data yang boleh dibaca. Nilai saldo hanya dapat dibaca dan tidak dapat diubah langsung, karena perubahan saldo hanya terjadi melalui method *setorTunai()* dan *tarikTunai()*.
- 4) *otentikasi(String pinInput)*
Method ini mengembalikan nilai *true* jika *PIN* yang diinput sama dengan *PIN* yang tersimpan dan *false* jika berbeda.
- 5) *Method setorTunai()*, *tarikTunai()*, *cekInformasi()*, dan *cetakMutasi()* tetap dipertahankan seperti pada sebelumnya yaitu Praktikum 2.

Pertanyaan:

Mengapa Tidak Membuat *setSaldo()* dan *getPin()*?

1. *setSaldo()*

Kalau dibuat *setSaldo()*, maka saldo dpat diubah sembarangan tanpa lewat aturan yang sudah dibuat (misalnya nominal harus lebih dari 0,

saldo harus cukup) dan tidak tercatat di riwayat transaksi. Makanya saldo hanya boleh berubah lewat *setorTunai()* dan *tarikTunai()*.

2. *getPin()*

Karena *pin* berseifat rahasia, sehingga dibuat fitur untuk mengecek atau *otentikasi()*, yang dapat menjawab benar atau salah tanpa membocorkan PIN aslinya.

B. TUGAS DAN LATIHAN

a) Mengamankan *CLI* dengan Otentikasi

1. Menu 1 (Buka Rekening) dengan Input PIN

Keterangan :

- 1) Sebelum objek *Rekening* dibuat, sistem meminta *user* memasukkan *PIN* (*String 6 digit*) dan menyimpannya pada variabel *pinBaru*.
- 2) *input.nextLine()* setelah *input.nextDouble()* diperlukan untuk membersihkan sisa enter, agar *PIN* tidak terbaca kosong.
- 3) *pinBaru* diteruskan sebagai argumen keempat pada *constructor Rekening_2511532002*.

Syntax :

```
switch (pilihan) {
case 1:
    System.out.println("Masukkan No Rekening: ");
    String no =input.nextLine();
    System.out.println("Masukkan Nama Pemilik: ");
    String nama =input.nextLine();
    System.out.println("Masukkan Saldo Awal: ");
    double saldo=input.nextDouble();
    input.nextLine();
    System.out.println("Masukkan PIN anda! (6 Digit)");
    String pinBaru=input.nextLine();

    //Instalasi Object/ Menjalankan Constructor
    akunAktif =new Rekening_2511532002(no, nama, saldo, pinBaru);
    daftarRekening.add(akunAktif);
    break;
```

Uji Coba :

1) Pembuatan rekening dengan PIN valid (6 digit)

```
=== SISTEM PERBANKAN MINI ===  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
0. Keluar  
Pilih Menu:  
1  
Masukkan No Rekening:  
11  
Masukkan Nama Pemilik:  
Haliya_2511532002  
Masukkan Saldo Awal:  
12000  
Masukkan PIN anda! (6 Digit)  
123456  
Rekening atas nama Haliya_2511532002 berhasil dibuat dengan saldo Rp12000.0
```

Dapat dilihat dari hasil uji coba tersebut bahwa fitur pin berhasil dibuat, pin diterima dan tersimpan untuk akun Haliya_2511532002.

2) Pembuatan rekening dengan PIN tidak valid

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
0. Keluar  
Pilih Menu:  
1  
Masukkan No Rekening:  
22  
Masukkan Nama Pemilik:  
Hal  
Masukkan Saldo Awal:  
150000  
Masukkan PIN anda! (6 Digit)  
12  
Peringatan: PIN Harus 6 digit! Menggunakan PIN Default 123456  
Rekening atas nama Hal berhasil dibuat dengan saldo Rp150000.0
```

Dapat dilihat padahal di atas jika *user* memasukan pin yang tidak sesuai maka akan muncul peringatan.

2. Keamanan Transaksi:

Menu 3 (Tarik Tunai)

Keterangan :

- 1) Sebelum meminta nominal penarikan, sistem meminta user memasukkan PIN dan memanggil *akunAktif.otentikasi(pinInput)*.
- 2) Jika mengembalikan nilai *true*, sistem meminta *nominal* dan menjalankan *tarikTunai()*.
- 3) Jika mengembalikan nilai *false*, transaksi dibatalkan dan dicetak pesan “*Akses Ditolak: PIN yang Anda masukkan salah!*”.

Syntax :

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error Mohon maaf, Anda belum memiliki nomor rekening!");
    } else {
        System.out.println("Masukkan PIN anda:");
        String pinInput = input.nextLine();
        if (akunAktif.otentikasi(pinInput)) {
            System.out.println("Masukkan nominal tarik: ");
            double tarik = input.nextDouble();
            input.nextLine();
            akunAktif.tarikTunai(tarik); //Memanggil behavior/ method
        } else {
            System.out.println("PIN Anda Salah!");
        }
    }
}break;
```

Uji coba :

- 1) Penarikan tunai dengan *PIN* yang benar

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu:
3
Masukkan PIN anda:
123456
Masukkan nominal tarik:
10000
Tarik tunai Rp10000.0 berhasil. Saldo saat ini: Rp140000.0
```

Dapat dilihat pada hasil berikut, bahwa penarikan berhasil dilakukan setelah memasukkan pin yang sesuai.

2) Penarikan tunai dengan *PIN* yang salah

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu:
3
Masukkan PIN anda:
12
PIN Anda Salah!
```

Dapat dilihat dari uji coba tersebut, bahwa penarikan gagal dilakukan jika yang dimasukkan salah.

Menu 6 (Cetak Mutasi)

Keterangan :

- 1) Sebelum mencetak mutasi, sistem meminta user memasukkan PIN dan memanggil *akunAktif.otentikasi(pinInput)*.
- 2) Jika mengembalikan nilai *true*, sistem menjalankan *cetakMutasi()*.
- 3) Jika mengembalikan nilai *false*, pencetakan dibatalkan dan dicetak pesan “Akses Ditolak: PIN yang Anda masukkan salah!”.

Syntax :

```
case 6:
    if (akunAktif==null)
        System.out.println("Anda belum membuka Rekening");
    else {
        System.out.println("Masukkan PIN anda:");
        String pinInput=input.nextLine();
        if (akunAktif.otentikasi(pinInput)) {
            akunAktif.cetakMutasi();
        }else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah! ");
        }
    }break;
```


Uji Coba :

- 1) Cetak mutasi dengan *PIN* yang benar

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
0. Keluar  
Pilih Menu:  
6  
Masukkan PIN anda:  
123456  
--- Cetak Mutasi ---  
ID: TRX-T-1790487739803|Jenis: Debit|Nominal: Rp10000.0
```

Dapat dilihat bahwa *user* berhasil melakukan cetak mutasi setelah memasukan *pin* yang benar.

- 2) Cetak mutasi dengan *PIN* yang salah

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi (Riwayat)  
0. Keluar  
Pilih Menu:  
6  
Masukkan PIN anda:  
12  
Akses Ditolak: PIN yang Anda masukkan salah!
```

Dapat dilihat dari hasil berikut bahwa dengan memasukan *PIN* yang salah, sistem menolak akses dan riwayat transaksi tidak ditampilkan.

Dari semua uji coba di atas yang telah dilakukan menunjukkan bahwa validasi *PIN* pada *constructor* serta *otentikasi* pada *Menu 3* dan *Menu 6* berfungsi sesuai ketentuan. Transaksi hanya dijalankan apabila *PIN* yang dimasukkan benar.

C. LINK REPOSITORY GITHUB

Seluruh kode program pada Praktikum 3, yaitu *class Transaksi_2511532002*, *class Rekening_2511532002*, dan *class Main_2511532002*, telah diunggah ke repository *GitHub*.

Berikut merupakan *link* dari repository *PrakPBO_B_2511532002*:

https://github.com/haliyaismaa/PrakPBO_B_2511532002/tree/main/PrakPBO_B_2511532002

Untuk file kode program dan juga laporan dari praktikum pekan ke 3 dapat diakses pada *path* *PrakPBO_B_2511532002/src/Pekan3_2511532002*.